

操作系统实验文档

Lab 6 - 内核页表 Kernel Paging

c476a06b

Built by github-actions, at 2025-03-25 14:41:56+08:00

1. 每周实验

1.1 RISC-V 页表模型 & xv6 内核页表

1.1.1 实验目的

1. 了解RISC-V页表模型SV39
2. 掌握虚拟地址到物理地址的转换机制
3. 掌握XV6如何管理内存并创建页表

1.1.2 Segmentation

从对内存访问保护而言，我们希望对内存的访问是带有权限保护的。这包括两个层面：

1. 该内存地址是否可读、可写、可执行。
2. 该内存地址是否允许低特权级访问。

也就是说，对于每个内存地址，我们都希望能检查它的操作权限是否符合原先的程序设计。

在 CPU 实现上，内存是以字节为单位来寻址的。如果要对每个字节的访问权限都能进行管理，这样的代价是难以想象的。但是，我们可以让相同权限的代码、数据排布在一起，这样即可将整个程序的内存空间分为几个大块，每个块均有自己的起始地址(Base)和大小限制(Limit)，以及权限设置，这就是使用 Segmentation 进行内存保护的方式。

Segmentation 在内存空间的管理上有着诸多劣势，例如要求物理内存连续、难以动态调节大小、存在碎片化的问题。所以，现代 CPU 和操作系统均使用页表（Paging）机制来实现内存管理。

1.1.3 Paging

分页机制，是将程序空间（虚拟地址）切割成相同大小的若干个页面，同时将物理内存也切割成同样大小的多个页面，从而可以在物理地址不连续的情况下，给进程分配足够的内存空间，并且从虚拟地址的角度看这个空间是连续的。

为什么需要虚拟地址空间？

如果我们只有物理内存空间，那么我们也可以写程序，但是所有的程序，包括内核，包括用户程序，都在同一个地址空间里，用户程序访问的 `0x80200000` 和内核访问的 `0x80200000` 是同一个地址。这样好不好？如果只有一个程序在运行，那也无所谓。但很多程序使用同一个内存空间，就会有问题：怎样防止程序之间互相干扰，甚至互相搞破坏？

比较粗暴的方式就是，我让用户程序访问的 `0x80200000` 和内核访问的 `0x80200000` 不是一个地址。但是我们只有一块内存，为了创造两个不同的地址空间，我们可以引入一个“翻译”机制：程序使用的地址（虚拟地址）需要经过一步“翻译”才能变成真正的内存的物理地址。这个“翻译”过程，我们用一个“词典”（页表）实现---给出翻译之前的地址，可以在词典里查找翻译后的地址。

每个程序都有唯一的一本“词典”，而它能使用的内存也就只有他的“词典”所包含的。

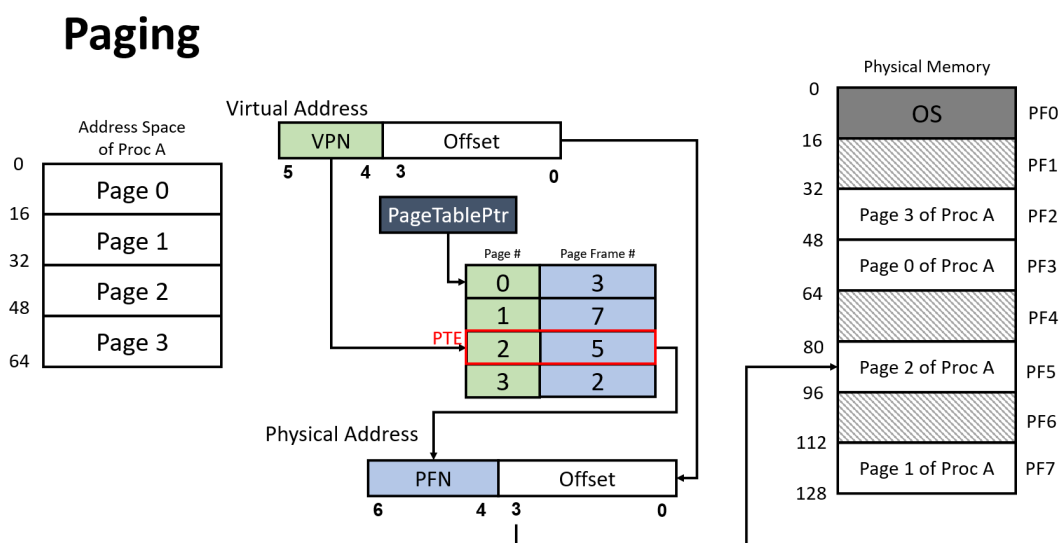
“词典”是否对能使用的每个字节都进行翻译？我们可以想象，存储每个字节翻译的结果至少需要一个字节，那么使用 1MB 的内存将至少需要构造 1MB 的“词典”，这效率太低了。观察到，一个程序使用内存的数量级通常远大于字节，至少以 KB 为单位（所以上古时代的人说的是“640K 对每个人都够了”而不是“640B 对每个人都够了”）。那么我们可以考虑，把连续的很多字节合在一起翻译，让他们翻译前后的数值之差相同，这就是“页”。

1.1.4 虚拟地址 => 物理地址

分页机制很重要的一点是如何建立和解析虚拟地址到物理地址的映射，下面我们从“如何从虚拟地址获得相应的物理地址”的角度进行介绍：

一级页表

如图所示是一个一级页表分页机制（一级对应于后面的多级，一级页表只需要查询一层页表即可得到物理地址，下称“单级页表”）：



以上图为例，我们首先得到一个 **虚拟地址 (Virtual Address)**，这个地址长度为6位，其中 5 ~ 4 位（高2位）为 **页号 (VPN, Virtual Page Number)**，3 ~ 0 位（低4位）为 **偏移量 (Offset)**。

通过虚拟地址，我们可以查询 **页表 (Page Table)**，页表存在于 **页表基地址 (PageTablePtr, Page Table Pointer)**，是个物理地址) 所指向的内存空间中，由连续存储的若干个 **页表项 (PTE, Page Table Entry)** 构成。在一级页表中，每个页表项内容即为 **物理页号 (Page Frame #) + 部分标志位**。尽管图中每个页表项看似包含 **页号 (Page #)** 但是在实际的设计中，**页号(VPN)并不写在页表项(PTE)中**，由于页表项是连续分布的，我们只需要知道页表项的大小（有多少位）以及虚拟地址页号（VPN，代表要查询第几个页表项），就可以通过 **页表首地址 + 页号 × 页表项大小** 得到对应的页表项地址，查询该地址对应内容即可得到物理页号。页表首地址是存储在架构指定的寄存器中的。

得到物理页号后，通过 **物理页号 × 页面大小** 即可得到所在页的物理地址（物理空间首地址为0x0）。一页可能很大，如何得到一页中具体某个字节的地址呢？通过偏移量，**页物理地址 + 偏移量** 即可得到具体虚拟地址对应的具体物理地址。

一个例子

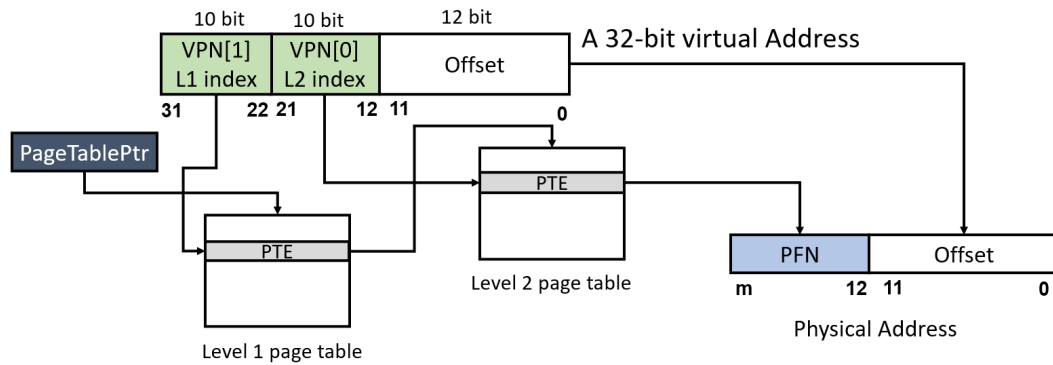
假设页面大小为 4KiB，如果想要定位到页面的每一个 Byte，我们的偏移量则需能表示 4096 个不同的位置，因此对于 4KiB 大小的页面来说，偏移量的位数为12位 ($2^{12}=4096$)。或者说，偏移量的位数为12位可以推出页面大小为 2^{12} 个 Byte。

在图片的一级页表中，Offset 为4位，可以得知页面大小为 $2^4=16\text{B}$ 。如果给定虚拟地址 100100_2 ，可以得出其页号为 10_2 ，即 2_{10} ，通过页号 2_{10} 可以查到对应的页表项内容为 5_{10} 。5是物理页号，可以求得物理页面地址：物理首地址 + 物理页号 × 页面大小 = $0 + 5 \times 16 = 80_{10}$ 。100100中低四位偏移量为0100，因此查询的是该页面的第5个字节对应的地址（地址0也是一个字节），则该物理地址为 $80 + 4 = 84_{10}$ ($5 \times 16 + 4 = 0101$ 左移四位 + $0100 = 0101\ 0100_2$)。需要注意运算过程中是2进制还是10进制还是16进制数（存储肯定是2进制）。

多级页表

系统中，通常页面大小是 4KiB。假设 32 位系统中，虚拟地址空间长度为 32bits，即 $2^{32} = 4\text{GiB}$ ，那么在单级页表模式下，就需要1M个页表项来对应不同的物理页。假设一个页表项为4B，则一个页表就有 4MiB。由于虚拟地址是连续的（相当于高位 VPN 是连续的），而PTE的存储方式也是相当于数组的连续存储方式，因此即使进程实际使用的空间非常小，它也需要连续完整的页表来进行地址转换（不能移除中间不使用的 PTE）。而操作系统中除了内核页表，为每个进程还会分配自己的页表，进程多的情况下，存储所有页表的开销就变得很大。

因此需要用到多级页表，如图所示，是一个二级页表分页机制 (Sv32)：



与单级页表不同的是，Sv32 将虚拟地址分割成了三个部分：VPN[1], VPN[0], offset。Sv32 中，每个页面大小 4KiB，每个 PTE 4 Bytes，每个页表中有 $2^{10} = 1024$ 个 PTE。

图中，32位虚拟地址的翻译过程变成了如下流程：

1. 通过 `PageTablePtr + VPN1 * 4` 得到第一级PTE地址（`PageTablePtr`存储于指定寄存器中）。
2. 取得第一级 PTE 中的PFN，它代表着第二级页表的基地址，通过 `PFN * 4Ki` 得到第二级页表的基地址。
3. 通过 `第二级页表基地址 + VPN0 * 4` 可以得到第二级PTE的地址
4. 取得第二级 PTE 中的PFN，它代表着最终的物理页面。通过 `PFN * 4Ki + offset` 可以得到最终的物理地址。

思考

为什么多级页表相比单级页表可以节省页面开销呢？

假设我们有一个 4GiB 的内存空间，页面大小为 4KiB，且每个页表项（PTE）的大小为 4Byte。如果一个进程只需要使用高虚拟地址空间 (0xffff_f000) 的 1 页 4KiB 空间和低虚拟地址空间 (0x0000_0000) 的 1 页 4KiB 空间，我们可以通过以下计算，比较单级页表和二级页表机制下，进程所需的页表空间。

单级页表：在单级页表的情况下，尽管我们只需要第0个和最后一个PTE，但是还是需要 $4\text{GiB} / 4\text{KiB} = 1\text{M}$ 个 **连续** 的页表项来映射整个虚拟地址空间。因此，单级页表机制所需要的页表大小是 $1\text{M} * 4\text{Byte} = 4\text{MiB}$ 。

多级页表：在多级页表机制中，每小一级页表所能表示的地址空间大小是逐级增加的。以 Sv32 为例，第二级页表中的每个 PTE 管理着 4KiB 的空间，那么整个第二级页表能管理 $1024 * 4\text{KiB} = 4\text{MiB}$ 的空间；第一级页表中的每个 PTE 管理着一张第二级页表，也就是说，第一级中的每个 PTE 实际上能管理 4 MiB 的空间。

对于地址空间中大片的、连续的空洞，我们可以直接不为他分配二级页表，这就是为什么多级页表可以节约空间。

对于上述情况，我们只需要在一级页表中分配两个 PTE，分别管理两个第二级页表；在第一个第二级页表中，我们用第 0 个 PTE 映射虚拟地址 0x0000_0000；在第二个第二级页表中，我们用第 1023 个 PTE 映射虚拟地址 0xffff_f000。

总结：为了管理这两个虚拟页面，二级页表需要 2 个二级页表和 1 个第一级页表（其余未用到的二级页表的 PTE 可以标记为不可用，无需分配空间）。由于第一级和第二级页表的大小均为 4KB，因此总共需要 $2 * 4\text{KB} + 4\text{KB} = 12\text{KB}$ 的页表空间。

通过这个例子可以看出，二级页表机制通过层级化的管理，显著减少了页表的空间开销，尤其是在只需要映射少量内存时。

注意

将虚拟地址翻译成物理地址的工作是由 CPU 的内存管理单元（MMU，Memory Management Unit）完成的，并不需要我们编写代码进行操作。

我们需要在操作系统中完成的是“词典”也就是页表的编写工作，为词条分配页表空间并填写页表项才是我们的工作。

快表 TLB

物理内存的访问速度要比 CPU 的运行速度慢很多，去访问一次物理内存可能需要几百个时钟周期（带来所谓的“冯诺依曼瓶颈”）。

而我们的页表都是存放在物理内存中。如果我们按照页表机制一步步走，将一个虚拟地址转化为物理地址需要访问 3 次物理内存，得到物理地址之后还要再访问一次物理内存，才能读到我们想要的数。这很大程度上降低了效率。

好在，实践表明虚拟地址的访问具有时间局部性和空间局部性。

- 时间局部性是指，被访问过一次的地址很有可能不久的将来再次被访问；
- 空间局部性是指，如果一个地址被访问，则这个地址附近的地址很有可能在不远的将来被访问。

因此，在 CPU 内部，我们使用 **快表 (TLB, Translation Lookaside Buffer)** 来记录近期已完成的虚拟页号到物理页号的映射。由于局部性，当我们要做一个映射时，会有很大可能这个映射在近期被完成过，所以我们可以先到 TLB 里面去查一下，如果有的话我们就可以直接完成映射，而不用访问那么多次内存了。

但是，我们如果修改了根页表寄存器的值，比如将上面的 PPN 字段进行了修改，说明我们切换到了一个与先前映射方式完全不同的页表。此时快表里面存储的映射结果就跟不上时代了，很可能是错误的。这种情况下我们要使用特定的指令刷新整个 TLB。在 RISC-V 中这个指令是 `sfence.vma`。

同样的，如果我们手动修改一个页表项，也等于修改了映射，但 TLB 并不会自动刷新，我们也需要使用特定的指令刷新 TLB。

在 RISC-V 中，如果不加参数，`sfence.vma` 会刷新整个 TLB。如果加上一个虚拟地址，则 `sfence.vma` 只会刷新这个虚拟地址的映射。

1.1.5 RISC-V 的 Sv39 分页机制

satp

satp (Supervisor Address Translation and Protection) 寄存器是控制 S mode 和 U mode 下地址翻译的寄存器，其中包含三个属性：`MODE`，`ASID` 和 `PPN`。

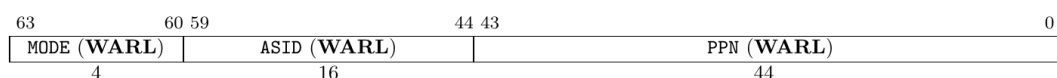


Figure 4.15: Supervisor address translation and protection register `satp` when `SXLEN=64`, for `MODE` values Bare, Sv39, Sv48, and Sv57.

Mode 表示使用的地址翻译模式，0 则表示禁用地址翻译，所有请求的地址均作为物理地址看待，`PPN` 表示 **根页表的基地址**。在我们的课程中，我们将使用 **Sv39** 作为页表模式。

我们暂且不需要理解 ASID 的作用。

SXLEN=32		
Value	Name	Description
0	Bare	No translation or protection.
1	Sv32	Page-based 32-bit virtual addressing (see Section 4.3).
SXLEN=64		
Value	Name	Description
0	Bare	No translation or protection.
1-7	—	<i>Reserved for standard use</i>
8	Sv39	Page-based 39-bit virtual addressing (see Section 4.4).
9	Sv48	Page-based 48-bit virtual addressing (see Section 4.5).
10	Sv57	Page-based 57-bit virtual addressing (see Section 4.6).
11	<i>Sv64</i>	<i>Reserved for page-based 64-bit virtual addressing.</i>
12-13	—	<i>Reserved for standard use</i>
14-15	—	<i>Designated for custom use</i>

Table 4.4: Encoding of **satp** MODE field.

Sv39

RISC-V 的 Sv39 模式支持了 39-bit 的虚拟地址空间，每个页面大小 4KiB，每个 PTE 8 bytes，即每个页面中有 $2^9=512$ 个 PTE。Sv39 要求每个页表是一个对齐的页面，即其基地址对齐到 4KiB。

RISC-V CPU 的虚拟地址为 64 位。Sv39 模式下，有效的虚拟地址为 39 位，并规定虚拟地址的 63-39 位必须与第 38 位相同，即符号扩展(Signed-Extension)，否则是一个非法的虚拟地址。所以，Sv39 的虚拟地址空间一共为 $(1 \ll 39) = 512 \text{ GiB}$ 的空间，其中分为高地址和低地址各 256 GiB 的空间，低地址空间为 `0x0000_0000_0000_0000 - 0x0000_003f_xxxx_xxxx`，而高地址空间为 `0xffff_ffc0_0000_0000 - 0xffff_ffff_xxxx_xxxx`。

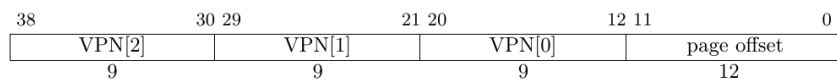


Figure 4.19: Sv39 virtual address.

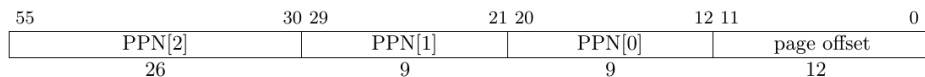


Figure 4.20: Sv39 physical address.

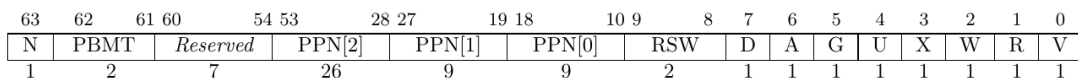


Figure 4.21: Sv39 page table entry.

虚拟地址分为四部分：VPN[2-0] (Virtual Page Number) 和 page offset。三级 VPN 表示在三级页表中的 index, 而 page offset 表示当前地址在被翻译的页面中的偏移量。

Sv39 中的 PTE 长度为 8-byte，分为两部分：PPN 和 Flags。PPN (Physical Page Number) 和虚拟地址中的 page offset 组成最终的物理地址，Flags 则表示该虚拟地址页面的访问权限等信息。

Flags

Flags 定义如下：

- D, A: Dirty, Accessed。表示该页面最近被访问 / 写入过。
- G: Global。表示该映射关系在所有页表中均存在。
- U: User。表示该映射关系允许在用户权限下访问。
- V: Valid。该 bit 表示此 PTE 为有效 PTE，否则整个 PTE 视为无效。
- R, W, X: Read, Write, Executable 权限

RWX 定义如下图所示：注意 `XWR == 3'b000` 的情况表示物理地址 [PPN: 12b0] 为下一级页表的基地址，否则，该PTE为 Leaf PTE.

X	W	R	Meaning
0	0	0	Pointer to next level of page table.
0	0	1	Read-only page.
0	1	0	<i>Reserved for future use.</i>
0	1	1	Read-write page.
1	0	0	Execute-only page.
1	0	1	Read-execute page.
1	1	0	<i>Reserved for future use.</i>
1	1	1	Read-write-execute page.

Table 4.5: Encoding of PTE R/W/X fields.

A & D

每个页 PTE 包含 Accessed 和 Dirty 两个 bits:

- A 表示: 自从上次 A bit 被清零, 该虚拟页面曾经被读取、写入、取指 (Instruction Fetch)。
- D 表示: 自从上次 D bit 被清零, 该虚拟页面曾经被写入。

当访问的虚拟页面被访问时, A bit 是 0、或被写入时, D bit 是 0 是, RISC-V 规范实现允许两种方式来更新 A & D bits:

1. 发出 PageFault, Supervisor 的异常处理函数需要手动设置 A / D bits.
2. 由硬件设置 A / D bits.

Supervisor 软件应当正确处理以上两种情况。

权限检查

凭直觉的, 读取的页面要带有 R bit, 写入的页面要带有 W bit, 执行的页面要带有 X bit。

但是, 如果一个页面的权限带有 U bit, 并且现在 CPU 核心运行在 S mode 下, 我们需要对 SUM (permit Supervisor User Memory access) bit 进行额外检查: 如果 `sstatus.SUM == 1` 则访问被允许, 否则导致 Page Fault.

通常来说, S mode 一般运行在 `sstatus.SUM == 0` 的情况下, 如果我们需要通过页表去访问用户数据时, 我们会将该 flag 置 1, 并在访问结束后清零。该过程一般被称为 **uaccess** 原语 (primitive).

See also: <https://github.com/torvalds/linux/blob/master/arch/riscv/include/asm/uaccess.h>

Sv39 地址翻译的过程如下图所示: (L2, L1, L0 分别代表 VPN2, VPN1, VPN0, 图中的 Page Directory 中, 下方为低地址, 上方为高地址)

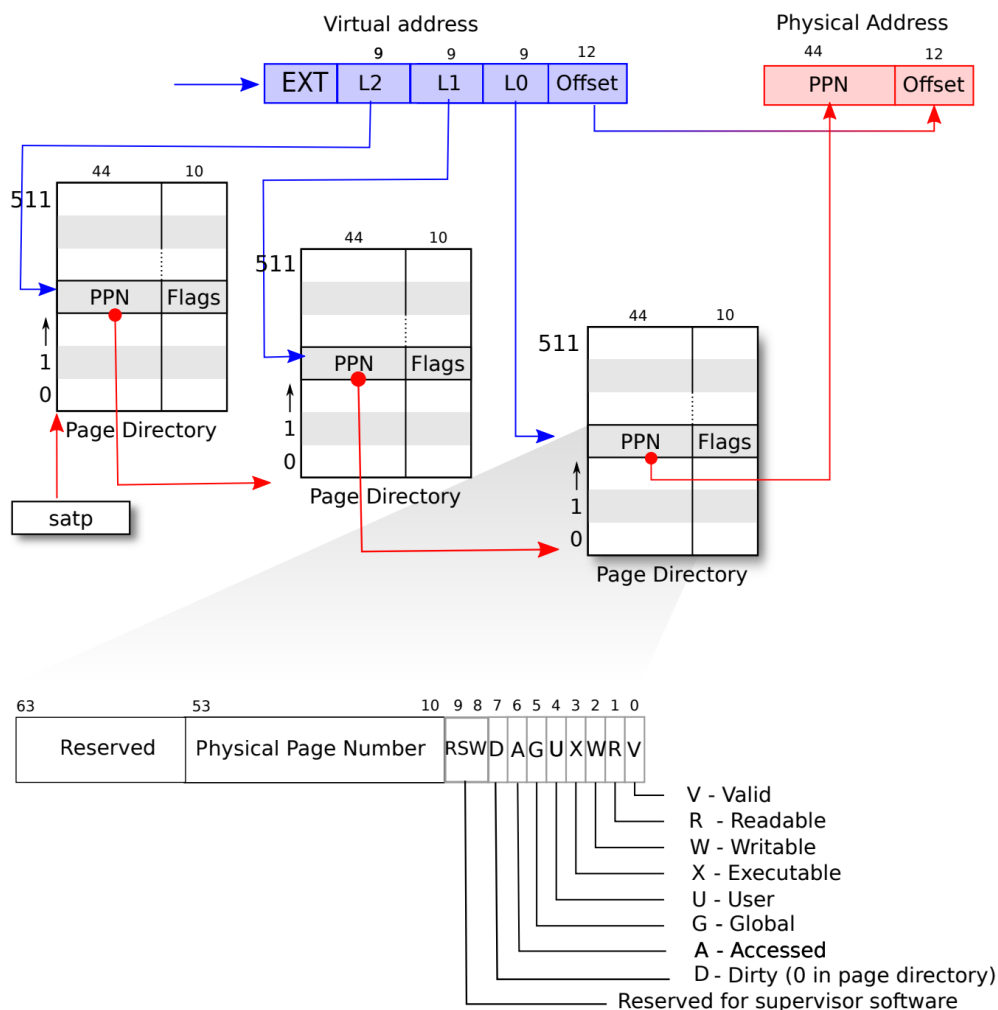


Figure 3.2: RISC-V address translation details.

See also: [riscv-privilege.pdf](#), 4.3.2 Virtual Address Translation Process

文字描述，以下 $\{xx \mid yy\}$ 表示在 xx bit 右边并上 yy bit，类似于 Verilog 的写法。

1. 分解 Virtual Address: $\{25'\text{signed_ext}, 9'\text{VPN2}, 9'\text{VPN1}, 9'\text{VPN0}, 12'\text{pgoff}\} = 64'\text{VirtualAddress}$

2. 将 satp 寄存器中的 PPN 作为第一级页表的基地址

3. 使用 VPN2 作为 index 在第一级页表中找到 pte2。

这一步等效于 C 代码: `uint64 pte2 = *(uint64*)(satp.base + VPN2 * 8);`

1. 如果 `pte2.WXR != 3'b000`，则表示该 PTE 为 1GiB 大页映射。

检查 PPN 是否对齐到 1GiB, aka: `pte2.PPN1 == 9'b0 && pte2.PPN0 == 9'b0`。如果满足, 则跳转至 10, 否则 Page Fault。

1. `{pte2.PPN, 12'b0}` 为第二级页表的基地址
2. 使用 VPN1 作为 index 在第二级页表中找到 pte1。

这一步等效于 C 代码: `uint64 pte1 = *(uint64*)((pte2.ppn << 12) + VPN1 * 8);`

1. 如果 `pte1.WXR != 3'b000`, 则表示该 PTE 为 2MiB 大页映射。

检查 PPN 是否对齐到 2MiB, aka: `pte2.PPN0 == 9'b0`。如果满足, 则跳转至 10, 否则 Page Fault。

1. 否则, `{pte1.PPN | 12'b0}` 为第三级页表的基地址
2. 使用 VPN0 作为 index 在第三级页表中找到 PTE。

这一步等效于 C 代码: `uint64 pte0 = *(uint64*)((pte1.ppn << 12) + VPN0 * 8);`

1. 得到最终的物理地址: `PA = (leaf_pte.ppn << 12) | final_page_offset`,

如果为 2MiB 大页映射, `final_page_offset = {9'VPN0, 12'pgoff}`。

如果为 1GiB 大页映射, `final_page_offset = {9'VPN1, 9'VPN0, 12'pgoff}`

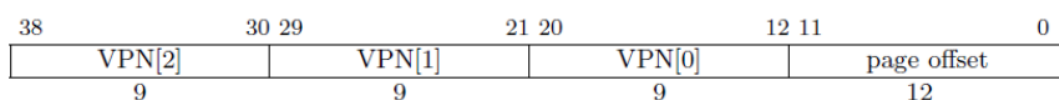
否则, `final_page_offset = 12'pgoff`

2. 权限检查: 检查 `leaf_pte.rwx` 是否与访存请求相同。

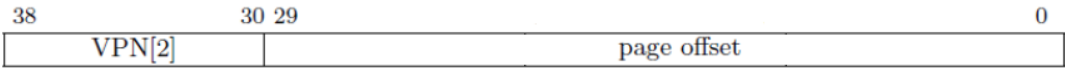
大页映射

可以注意到当 Flags 中的 XWR 取值均为0时, 该 PTE 是指向下一级页表的指针, 而当这 XWR 中存在非0位时, 则该PTE是页表树的一个叶节点, 也就是此时该PTE的PPN值代表了 **物理页** 的基地址而非 **页表** 的物理基址。

比如, Sv39中给定一个虚拟地址VA[38:0], 然后我们根据 `satp.PPN×4096+VPN2` 求得一级页表中的 PTE的地址, 假设该PTE的XWR为001, 则该PTE为叶节点页表项, 我们称之为大页映射。此时, 由于一级页表中的页表项就是叶节点页表项, 虚拟地址的含义就从原本的



变成了



而物理地址的计算则变成了 $PTE.PPN \times 4096 + VA[29:0]$ 。此时可以发现30位偏移量可以表示从 $PTE.PPN \times 4096$ 开始的一个很大的空间（空间大小为 2^{30} Byte即1GiB），我们称之为 **吉页**。（注意常见支持大页机制的架构中，大页基地址必须对齐自己的大页空间，即吉页基地址必须对齐1GB， $PTE.PPN \times 4096$ 的低30位需要为0。）

相应的，如果第二级页表的页表项为叶节点页表项，则该页表项的 `[20:0]` 位将表示一个整体的偏移量，代表一个小于吉页的页，我们称之为 **巨页**。一般情况下三级页表项是叶节点页表项，指向一个 **基页**（4KB）。

1.1.6 Lab 课堂报告

1. 请参照 Sv39 地址翻译模式，解码以下虚拟地址，写出其 VPN[2-0] 和 offset 的二进制值。**注意：请用二进制表示！**

vaddr	VPN2	VPN1	VPN0	offset
0x0000_0000_8020_1234				
0x0000_003f_ffff_f000				
0xffff_ffd1_dead_beef				

Hint: 你可以使用 Windows 中自带的计算器，切换到 Programmer 模式，即可快速在16、10、2进制间转换。或者可以使用 python3 自带的 `bin(0x1122_3344)` 函数转换数字到二进制。

1. 请你参照 Sv39 模式手动进行地址翻译，请你写出该页表中所有有效的映射。

satp: 0x80000000000080400

部分内存的 hexdump 如下，冒号左侧代表地址，右侧代表该地址和该地址+0x8处的两个 `uint64_t` 的16进制表示。未指定的内存均为0。hexdump已经转换为小端序，你不需要考虑端序问题。

```

0x8040_0000:
0x0000000080400000: 0x0000000020100401
0x00000000804004a8: 0x0000000020101401
0x00000000804007f8: 0x0000000020100c01

0x8040_1000:
0x0000000080401000: 0x0000000020100801

0x8040_2000:
0x0000000080402008: 0x000000002000001f
0x0000000080402010: 0x000000002000041f

```

```

0x8040_3000:
0x00000000080403ff8:      0x0000000020101001

0x8040_4000:
0x00000000080404ff8:      0x000000003777400b

0x8040_5000:
0x000000000804056f0:      0x00000037ab400043

```

注意：所有数字均以16进制表示，请参照表格中第一条的格式。

Virtual Address	Size	PPN	Flags(DAGUXWRV)
0xdead_beef_aabb_ccdd	0x00aa_eeff	0x8899_0000	DA-UWXRv

1. 在 RISC-V 平台上，页面大小为 `4KiB`。在 32 位下我们可用 Sv32，在 64 位下我们可用 Sv39。已知 Sv32 中 PTE 的大小为 4 Bytes，Sv39 中 PTE 的大小为 8 Bytes。

Sv32 page tables consist of 2^{10} page-table entries (PTEs), each of four bytes. A page table is exactly the size of a page and must always be aligned to a page boundary. The physical page number of the root page table is stored in the satp register.

Sv39 page tables contain 2^9 page table entries (PTEs), eight bytes each. A page table is exactly the size of a page and must always be aligned to a page boundary. The physical page number of the root page table is stored in the satp register's PPN field.

请你简要解释为什么 Sv32 中每个 VPN 的长度是 10 bits，而 Sv39 中每个 VPN 的长度是 9 bits。

1.1.7 实验场景

xv6-lab4 代码分支

<https://github.com/yuk1i/SUSTech-OS-2025/tree/xv6-lab4>

使用命令 `git clone https://github.com/yuk1i/SUSTech-OS-2025 -b xv6-lab4 xv6lab4` 下载 xv6-lab4 代码。

使用 `make run` 运行本次 Lab 的内核，内核会进行重定位启动，最终，它会运行在高地址的 pc 和 sp 上。

```

Kernel Starts Relocating...
Kernel size: 0x000000000002e000, Rounded to 2MiB: 0x000000000200000
[INFO 0,-1] bootcpu_start_relocation: Kernel phy_base: 0x0000000008020000, phy_end_4k:0x0000000008022e000, phy_end_2M
0x00000000080400000
Mapping Identity: 0x00000000080200000 to 0x00000000080200000
Mapping kernel image: 0xffffffff80200000 to 0x00000000080200000
Mapping Direct Mapping: 0xffffffffc080400000 to 0x00000000080400000
Enable SATP on temporary pagetable.
Boot HART Relocated. We are at high address now! PC: 0xffffffff80203d68
[INFO 0,-1] kvm_init: boot-stage page allocator: base 0xffffffffc080400000, end 0xffffffffc080600000
[INFO 0,-1] kvmmake: Memory after kernel image (phys) size = 0x0000000007c00000
[INFO 0,-1] kvm_init: enable pageing at 0x8000000000080400
[INFO 0,-1] kvm_init: boot-stage page allocator ends up: base 0xffffffffc080400000, used: 0xffffffffc080411000
Relocated. Boot halt sp at 0xffffffffffff001fb0

```

在本次的实验代码中，我们会为内核构建虚拟地址空间，并将操作系统从运行在物理地址空间跳转到运行在虚拟地址空间。

代码的具体执行流程如下：

1. entry.S 的 `_entry`：操作系统入口
2. main.c 的 `bootcpu_entry`：C语言入口
3. main.c 的 `bootcpu_start_relocation`：将 PC 和 satp 进行重定位，设置临时页表
4. main.c 的 `bootcpu_relocating`：调用 `kvm_init` 完成对内核页表的设置与切换
5. main.c 的 `bootcpu_init`：内核初始化，跳转 `scheduler`

要将运行在物理地址空间改为运行在虚拟地址空间，我们首先需要了解内核放置在物理空间的什么区域：

1.1.8 RISC-V 物理地址布局

首先，RISC-V 将物理内存 (DDR / DRAM) 的起始地址映射在物理地址 `0x8000_0000` 上，而不是物理地址 `0x0000_0000` 处。

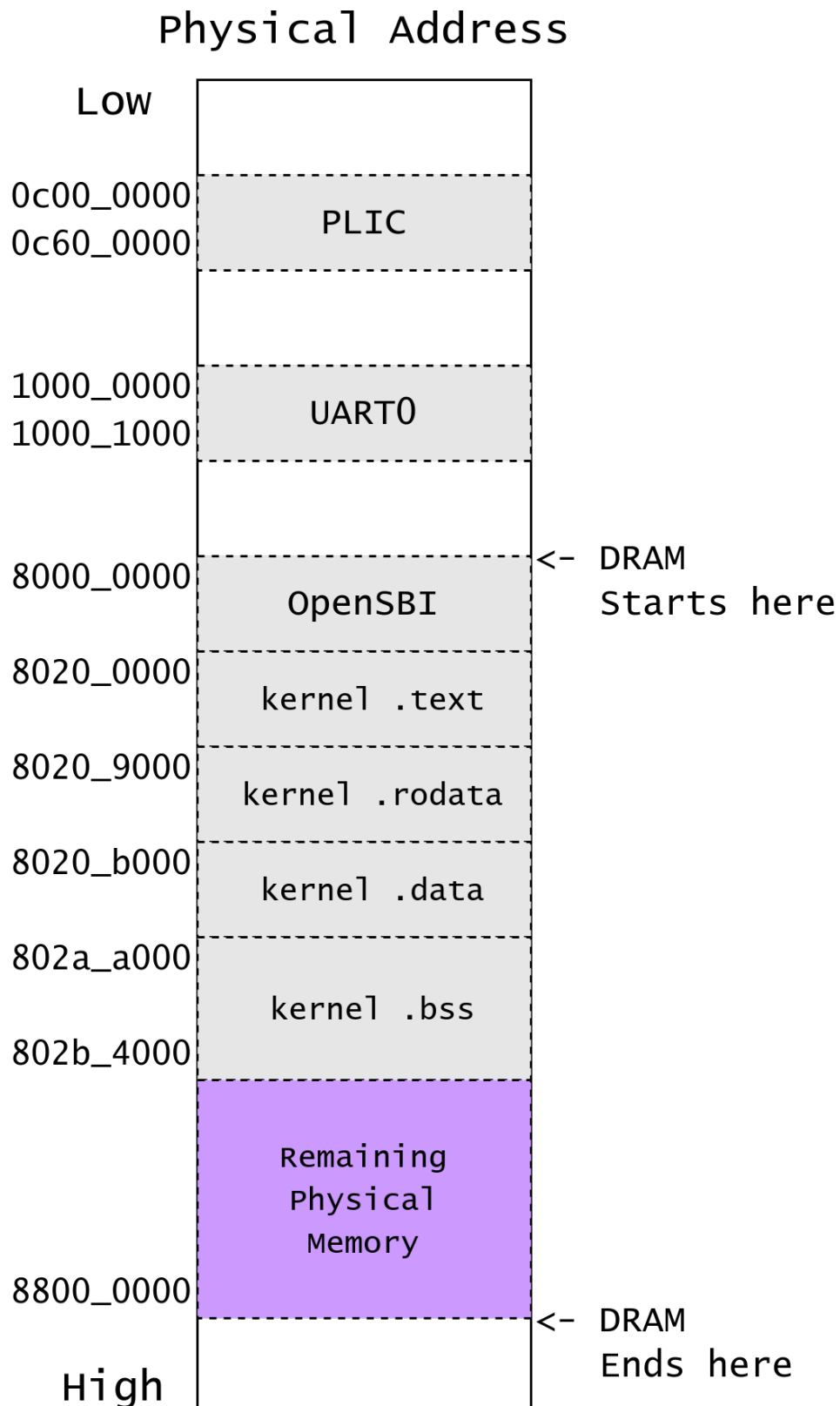
也就是说，如果我们有 128 MiB (`0x0800_0000`) 的 DRAM 大小，RISC-V 核心会将 DRAM 空间映射到 `[0x8000_0000, 0x8800_0000)` 上面。

对于 QEMU 的 virt 机子，我们可以翻阅 qemu 的源代码找到其物理地址的映射：

```
static const MemMapEntry virt_memmap[] = {
    [VIRT_DEBUG] = { 0x0, 0x100 },
    [VIRT_MROM] = { 0x1000, 0xf000 },
    [VIRT_TEST] = { 0x100000, 0x1000 },
    [VIRT_RTC] = { 0x101000, 0x1000 },
    [VIRT_CLINT] = { 0x2000000, 0x10000 },
    [VIRT_ACLINT_SSWI] = { 0x2f00000, 0x4000 },
    [VIRT_PCIE_PIO] = { 0x3000000, 0x10000 },
    [VIRT_PLATFORM_BUS] = { 0x4000000, 0x2000000 },
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
    [VIRT_APLIC_M] = { 0xc000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_APLIC_S] = { 0xd000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_UART0] = { 0x10000000, 0x100 },
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },
    [VIRT_FW_CFG] = { 0x10100000, 0x18 },
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },
    [VIRT_IMSIC_M] = { 0x24000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_IMSIC_S] = { 0x28000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },
    [VIRT_DRAM] = { 0x80000000, 0x0 },
};
```

不过，在我们的操作系统实验中，我们只需要关注 DRAM 空间和一些外设(PLIC, UART)即可

Base	Size	Description
0x0000_1000	0x0000_f000	BootROM
0x0c00_0000	0x0060_0000	PLIC
0x1000_0000	0x0000_0100	Serial UART
0x8000_0000	DRAM Size	DRAM



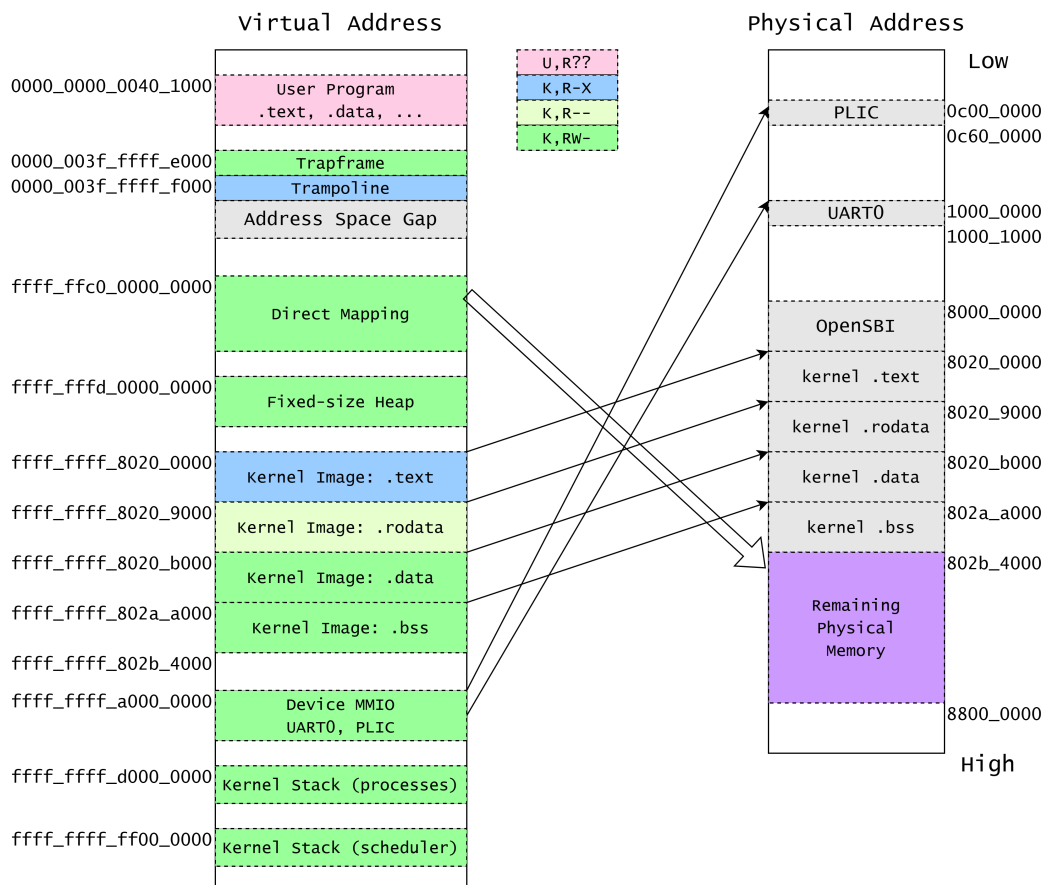
接着，我们需要设计一个内存布局，它将决定对于内核，我们将建立怎么样的地址映射。

1.1.9 xv6 内核内存布局

Sv39 虚拟地址的高位是 Sign-Extension 的，在 $< 256 \text{ GiB}$ 和 $256 \text{ GiB} \sim 512 \text{ GiB}$ 之间有着巨大的 gap，我们利用此特性在地址上区分用户地址（低，以 `0x0000` 开头）和内核地址（高，以 `0xffff` 开头）。

Base Address	Description
<code>0x0000_0000_xxxx_xxxx</code>	Userspace
<code>0x0000_003f_ffff_e000</code>	Trapframe
<code>0x0000_003f_ffff_f000</code>	Trampoline
<code>0xffff_ffc0_0000_0000</code>	Kernel Direct Mapping of all available physical pages
<code>0xffff_ffff_0000_0000</code>	Kernel Heap (fixed-size object)
<code>0xffff_ffff_8020_0000</code>	Kernel Image (.text, .data, .bss)
<code>0xffff_ffff_a000_0000</code>	Device Memory-Mapped IO
<code>0xffff_ffff_d000_0000</code>	Kernel stack for processes
<code>0xffff_ffff_ff00_0000</code>	Kernel stack for per-cpu scheduler

- Trampoline (n. 蹦床) 是用户空间和内核空间的跳板，我们将它放在低 128 GiB 的最高处。
- 然后，我们将内核的镜像，即 `build/kernel` ELF 文件，我们令它映射到虚拟地址 `0xffff_ffff_8020_0000`，与实际的物理地址 `0x8020_0000` 相差一个常数 `0xffff_ffff_0000_0000`。
- 其次，映射一些内核所需要的页面，如每个 CPU 的 scheduler 所用的栈，以及外设所需要的 MMIO。
- 最后，剩下的所有可用的物理页面将被 Direct Mapping 到 `0xffff_ffc0_80xx_xxxx`，并交给 `kalloc` 管理。



Direct Mapping

对于可用的物理内存空间，我们使用 Direct Mapping 的方式建立地址映射，即 $\text{虚拟地址} = \text{物理地址} + \text{常数偏移量}$ 。

Direct Mapping 的作用是让 Kernel 能直接操纵所有可用的物理内存，但是除了内核本身镜像以外。

如果没有 Direct Mapping，我们每次都需要将新分配的页面映射到内核虚拟地址空间上，才能通过虚拟地址去访问该物理页面。而有了 Direct Mapping 后，我们可以直接将物理地址加上一个常量偏移量，即可得到一个内核可访问的虚拟地址：

```
#define KERNEL_DIRECT_MAPPING_BASE 0xffffffc000000000ull

#define KVA_TO_PA(x) (((uint64)(x)) - KERNEL_DIRECT_MAPPING_BASE)
#define PA_TO_KVA(x) (((uint64)(x)) + KERNEL_DIRECT_MAPPING_BASE)
```

在设计完内核内存布局后，我们需要令内核从物理地址切换到虚拟地址继续运行，这一步需要首先做一个重定位的操作：

1.1.10 重定位 (Relocation)

对于内核本身（即编译出来的 ELF 文件：`build/kernel`，也称为内核镜像），一直是放置在 `0x8020_0000` 的物理地址上的。现在，我们需要把它映射到高地址上。我们将采用固定偏移量来映射整个 ELF 文件。

也就是说，内核中定义 (Defined) 的符号（变量、函数），它们会被 OpenSBI 加载到指定的物理地址 `0x0000_0000_8020_abcd`，而我们最终会将该符号加载到虚拟地址 `0xffff_ffff_8020_abcd` 上。对于所有符号，这两个地址之间永远相差一个固定的值。我们将该值定义为内核偏移量 (kernel offset)。

我们将该值定义为宏 `KERNEL_OFFSET`，并定义宏 `KIVA_TO_PA` 和 `PA_TO_KIVA` 在便于两者之间转换。(KIAV: Kernel Image Virtual Address)

```
#define KERNEL_VIRT_BASE 0xffffffff80200000ull
#define KERNEL_PHYS_BASE 0x80200000ull
#define KERNEL_OFFSET    ((uint64)(KERNEL_VIRT_BASE - KERNEL_PHYS_BASE))

// (Kernel Image Virtual Address) TO (Physical Address)
#define KIVA_TO_PA(x) (((uint64)(x)) - KERNEL_OFFSET)
#define PA_TO_KIVA(x) (((uint64)(x)) + KERNEL_OFFSET)
```

我们注意到，偏移整个镜像文件加载的基地址并不会改变两个符号之间的距离，也就是说：

1. 符号 a 被加载到 `PA_a 0x0000_0000_8020_dead`，符号 b 被加载到 `PA_b 0x0000_0000_8020_beef`。
2. 符号 a 将会被映射到 `VA_a 0xffff_ffff_8020_dead`，符号 b 将会被映射到 `VA_b 0xffff_ffff_8020_beef`
3. `PA_a - PA_b = VA_a - VA_b`

这个性质允许我们在使用 `PC-relative` 寻址时，对整个内核镜像进行重定位。使用 `PC-relative` 寻址时，我们会通过当前 PC 与目标符号的偏移量来计算目标符号的地址，具体而言，在汇编上使用 `auipc` 和 `addi` 来计算 `target = pc + offset`，而不是 `target = immediate`（立即数寻址）。所以，我们可以将整个内核镜像整体重定位到任何起始地址，只需要保证所有符号之间的偏移量在编译期间和被加载时是固定的即可。这一性质也是 Linux Kernel 能实现对自身镜像的 KASLR (Kernel address space layout randomization, See also: <https://lwn.net/Articles/569635/>) 的原理。

因此，我们只需要完成两步即可实现内核镜像的重定位：

1. 将内核镜像映射到虚拟地址 `0xffff_ffff_8020_0000` 上。
2. 将 PC 跳转到 `0xffff_ffff_8020_xxxx` 的高地址上。

我们预期内核会运行在 `0xffff_ffff_8020_0000` 的高地址上。所以，我们需要修改 `kernel.ld` 使链接器能正确链接内核。

```
OUTPUT_ARCH(riscv)
ENTRY(_entry)
BASE_ADDRESS = 0xffffffff80200000;
```

但是，此时 OpenSBI 无法正确加载 kernel ELF 到物理地址上，因为 kernel ELF 的 Program Header 的预计加载的物理地址 (PhysAddr) 是 `0xffff_ffff_8020_0000` 的高地址，然而这个地址在 OpenSBI 加载内核时是非法的。

所以，我们在第一个段 `.text` 后面注明 `AT(0x80200000)`，表示这个段应该被加载到物理地址 `0x8020_0000`。这样能使链接器产生的 ELF 的 Program Headers 拥有 `VirtAddr = 0xffff_ffff_8020_0000`，以及 `PhysAddr = 0x8020_0000`。

```
BASE_ADDRESS = 0xffffffff80200000;
SECTIONS
{
    . = BASE_ADDRESS;
    skernel = .;
    s_text = .;

    .text : AT(0x80200000) {
        *(.text.entry)
        // ...
    }
    // ...
}
```

此时再 `make run` 运行内核，我们可以发现 OpenSBI 正确找到了我们内核的入口点：

```
Domain0 Next Address      : 0x0000000080200000
Domain0 Next Arg1         : 0x000000009fe00000
Domain0 Next Mode         : S-mode
...
clean bss: 0x00000000802ac000 - 0x00000000802b3000
Kernel is Relocating...
```

接着，我们需要构建页表来让MMU可以按照我们设计的布局翻译我们的虚拟地址。

如果我们直接按照设计好的内核内存布局构建页表，我们将需要如下两条或更多指令来完成切换页表+跳转到高地址：

1. `csrw satp`：设置 `satp` 寄存器，启用 Sv39 地址翻译
2. ...

但是，当执行第一条指令时，我们的 PC 指向在 `0x8020_xxxx` 的物理地址上。可是，当我们执行完第 1 条指令设置完 `satp` 后，马上将开始执行第二条指令，而我们的 PC 还指向第二条指令的物理地址，

而这个物理地址的 PC 在当前的页表中是非法的。所以，我们的第二条指令就会触发 Instruction Page Fault 异常。

也就是说，在我们设置完内核页表后，我们并不能直接切换到仅包含高地址的页表上，因为此时我们的 PC 指针还指向低地址，而我们无法在同时完成切换 PC + 切换 satp 两件事情。所以，我们需要一个临时页表，其中包含了两部份映射：

1. VA 0x0000_0000_8020_0000 -> PA 0x8020_0000
2. VA 0xffff_ffff_8020_0000 -> PA 0x8020_0000

当执行完上述第一条指令启用 Sv39 后，我们目前的 PC 虽然是下一条指令的物理地址，但是它是一个合法的虚拟地址。我们可以加载一个立即数到寄存器中，然后使用 `jr` 指令跳转到该寄存器的值，从而进入到高地址。

我们将这一系列步骤称为 Relocate (重定位)。

See also: <https://0xax.gitbooks.io/linux-insides/content/Booting/linux-bootstrap-6.html>

relocation_start 代码解释

在 `relocation_start` 的临时页表中，我们使用 2 MiB 的大页映射。我们先开辟四个对齐的物理页面：

```
static char relocate_pagetable[PGSIZE] __attribute__((aligned(PGSIZE)));
static char relocate_pagetable_level1_idents[PGSIZE] __attribute__((aligned(PGSIZE)));
static char relocate_pagetable_level1_direct_mapping[PGSIZE] __attribute__((aligned(PGSIZE)));
static char relocate_pagetable_level1_high[PGSIZE] __attribute__((aligned(PGSIZE)));
```

我们计算内核镜像的终止点 `ekernel` 向上对齐到 2MiB 的大小。这是因为 2 MiB 的大页映射要求虚拟地址和物理地址同时对齐到 2MiB 上。

```
// Kernel Start Point must be aligned to 2MiB
assert(IS_ALIGNED(KERNEL_PHYS_BASE, PGSIZE_2M));

// Calculate Kernel image size, and round up to 2MiB.
uint64 kernel_size = (uint64)ekernel - (uint64)skernel;
uint64 kernel_size_4K = ROUNDUP_2N(kernel_size, PGSIZE);
uint64 kernel_size_2M = ROUNDUP_2N(kernel_size, PGSIZE_2M);

kernel_image_end_4k = KERNEL_PHYS_BASE + kernel_size_4K;
kernel_image_end_2M = KERNEL_PHYS_BASE + kernel_size_2M;

printf("Kernel size: %p, Rounded to 2MiB: %p\n", kernel_size, kernel_size_2M);
```

在 `kernel_image_end_2M` 后面，我们再开辟一个 2MiB 的页面作为 Kernel Direct Mapping 的第一个内存池，我们会在后续讲到为什么会这么做。

```
// Calculate Kernel Mapping Base & End
uint64 kernel_phys_base = KERNEL_PHYS_BASE;
uint64 kernel_phys_end = kernel_phys_base + kernel_size_2M;
uint64 kernel_virt_base = KERNEL_VIRT_BASE;
uint64 kernel_virt_end = kernel_virt_base + kernel_size_2M;

// Calculate the first Direct Mapping Base & End
uint64 kernel_la_phy_base = kernel_image_end_2M;
uint64 kernel_la_base = KERNEL_DIRECT_MAPPING_BASE + kernel_la_phy_base;
uint64 kernel_la_end = kernel_la_base + PGSIZE_2M;
```

然后，我们开始映射：VA `0x0000_0000_8020_0000` -> PA `0x8020_0000`

1. 在 `pgt_root` 上添加一条 PTE，使其指向第一级页表 `pgt_ident`。
2. 从 `kernel_phys_base` 到 `kernel_phys_end`，每 2 MiB 添加一个 PTE 映射
3. 计算该物理地址应该被映射到哪个虚拟地址上，在这个映射中，`va = pa`。
4. 计算 `VPN1`，并在 `pgt_ident` 中添加映射。

```
// We will still have some instructions executed on pc 0x8020xxxx before jumping to KIVA.
// Step 2. Setup Identity Mapping for 0x80200000 -> 0x80200000, using 2MiB huge page.
{
    uint64 VPN2 = PX(2, kernel_phys_base);
    pgt_root[VPN2] = MAKE_PTE((uint64)pgt_ident, 0);

    for (uint64 pa = kernel_phys_base; pa < kernel_phys_end; pa += PGSIZE_2M) {
        uint64 va = pa;
        uint64 vpn1 = PX(1, va);
        pgt_ident[vpn1] = MAKE_PTE(pa, PTE_R | PTE_W | PTE_X | PTE_A | PTE_D);
        printf("Mapping Identity: %p to %p\n", va, pa);
    }
}
```

然后，我们开始映射内核 ELF 的虚拟地址：VA `0xffff_ffff_8020_0000` -> PA `0x8020_0000`。此时 `va = pa + KERNEL_OFFSET`。

最后，我们映射第一块 Direct Mapping：VA `0xffff_ffc0_80xx_0000` -> 第一个空闲的 2 MiB 物理页 `0x80xx_0000`。

1.1.11 内核页表

code: `os/kvm.c`

在完成 Relocation 后，我们的 pc 位于高地址，sp 位于 `boot_stack` 的高地址：

```
Mapping Identity: 0x0000000080200000 to 0x0000000080200000
Mapping kernel image: 0xffffffff80200000 to 0x0000000080200000
Mapping Direct Mapping: 0xffffffffc080400000 to 0x0000000080400000
Enable SATP on temporary pagetable.
Boot HART Relocated. We are at high address now! PC: 0xffffffff80203d68, SP: 0xffffffff80223ff0
```

我们在 `bootcpu_relocating` 中调用 `kvm_init()` 来构造最终的内核页表，并切换到该页表上。

在构建内核页表时，我们会需要申请物理页面来放置页表，我们称之为 `boot-stage page allocator`，这些页面的生命周期是持久的，永远不会被释放。但是后续我们会需要访问这些页表，所以我们可以令它的物理地址区域位于 Direct Mapping 区间上，并且以后能通过 Direct Mapping 的虚拟地址进行访问。Relocation 时，我们会在 Direct Mapping 区间先借用一个 2MiB 的区域，即 `e_kernel` (`0x802x_0000`) 对齐到 2MiB (`0x8040_0000`) 后的下一个 2MiB 区域 [`0x8040_0000`, `0x8060_0000`)。

```
[INFO 0,-1] kvm_init: boot-stage page allocator: base 0xffffffffc080400000, end 0xffffffffc080600000
```

`kvmmake` 函数会调用 `kvmmap` 依次映射每个区域。

```
static pagetable_t kvmmake() {
    pagetable_t kpgtbl;
    kpgtbl = (pagetable_t)allocsetuppage();
    memset(kpgtbl, 0, PGSIZE);

    // Step.1 : Kernel Image
    // Step.2 : Kernel Trampoline
    // Step.3 : Kernel Device MMIO :
    // Step.4 : Kernel Scheduler stack:
    // Step.5 : Kernel Direct Mapping
}
```

`kvmmap` 的原型为 `void kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm)`，表示将 `[va, va+sz)` 映射到 `[pa, pa+sz)` 的区域上。

1. 对于 Kernel Image，我们映射三个分区：`.text` (RX), `.rodata` (RO), `.data` (`.bss`) (RW)。我们可以引用在 `kernel.ld` 中导出的地址符号 `e_text` 等来得到每个分区的起始地址和结束地址。注意我们直接引用符号 `s_text` 时，我们会通过 `auipc` 指令得到它的虚拟地址；而它的物理地址则可以通过 `KIVA_TO_PA` 转换得到。

在 `kernel.ld` 中的 `.text` 存在另一个特殊的页面，称为 `trampoline`，我们会在下一节课讲到它。我们将它映射到虚拟地址 `0x3f_ffff_f000` 上。

```
// map kernel text executable and read-only.
// 0xffff_ffff_8020_0000 -> 0x8020_0000
kvmmap(kpgtbl, (uint64)s_text, KIVA_TO_PA(s_text), (uint64)e_text - (uint64)s_text, PTE_A | PTE_R |
```



```

PTE_X);

// map kernel ro_data: s_rodata to e_rodata
kvmmmap(kpgtbl, (uint64)s_rodata, KIVA_TO_PA(s_rodata), (uint64)e_rodata - (uint64)s_rodata, PTE_A |
PTE_R);

// map kernel .s_data to .e_bss,
uint64 kimage_data_size = KIVA_TO_PA(e_bss) - KIVA_TO_PA(s_data);
kvmmmap(kpgtbl, (uint64)s_data, KIVA_TO_PA(s_data), kimage_data_size, PTE_A | PTE_D | PTE_R | PTE_W);

// map trampoline
kvmmmap(kpgtbl, (uint64)TRAMPOLINE, KIVA_TO_PA(trampoline), PGSIZE, PTE_A | PTE_R | PTE_X);

```

此时，`kernel_pagetable` 的结构如下所示：

```

=== PageTable at 0xffffffffc080400000 ===
[ff], pte[0xffffffffc0804007f8]: 0x00000003fc00000000 -> 0x00000000080403000 -A---V
[1ff], pte[0xffffffffc080403ff8]: 0x00000003fffe000000 -> 0x00000000080404000 -A---V
[1ff], pte[0xffffffffc080404ff8]: 0x00000003ffffff0000 -> 0x0000000008020a000 -A--X-RV
[1fe], pte[0xffffffffc080400ff0]: 0xfffffffff800000000 -> 0x00000000080401000 -A---V
[1], pte[0xffffffffc080401008]: 0xfffffffff802000000 -> 0x00000000080402000 -A---V
[0], pte[0xffffffffc080402000]: 0xfffffffff802000000 -> 0x00000000080200000 -A--X-RV
...
[9], pte[0xffffffffc080402048]: 0xfffffffff802090000 -> 0x00000000080209000 -A--X-RV
[b], pte[0xffffffffc080402058]: 0xfffffffff8020b0000 -> 0x0000000008020b000 -A---RV
...
[20], pte[0xffffffffc080402100]: 0xfffffffff802200000 -> 0x00000000080220000 -A---RV
[21], pte[0xffffffffc080402108]: 0xfffffffff802210000 -> 0x00000000080221000 DA---WRV
...
[2c], pte[0xffffffffc080402160]: 0xfffffffff8022c0000 -> 0x0000000008022c000 DA---WRV
=== END ===

```

1. 然后，我们映射外设的 MMIO 区域，目前我们会使用到 PLIC 和 UART0 两个外设。

```

// Step.3 : Kernel Device MMIO :
kvmmmap(kpgtbl, KERNEL_PLIC_BASE, PLIC_PHYS, KERNEL_PLIC_SIZE, PTE_A | PTE_D | PTE_R | PTE_W);
kvmmmap(kpgtbl, KERNEL_UART0_BASE, UART0_PHYS, KERNEL_UART0_SIZE, PTE_A | PTE_D | PTE_R | PTE_W);

```

1. 然后，我们映射每个 cpu 的 scheduler 所用的栈。我们令每个 scheduler 的内核为两个 `PGSIZE`，并且相邻的两个栈空间相隔一段非法的地址，我们称之为 `guard page`。如果我们在内核栈上发生栈溢出，我们会得到一个 Page Fault 而不是内核带着错误悄无声息的继续运行。

```

// Step.4 : Kernel Scheduler stack:
uint64 sched_stack = KERNEL_STACK_SCHED;
for (int i = 0; i < NCPU; i++) {
    struct cpu *c = getcpu(i);
    // allocate #KERNEL_STACK_SIZE / PGSIZE pages
    for (uint64 va = sched_stack; va < sched_stack + KERNEL_STACK_SIZE; va += PGSIZE) {
        uint64 __pa newpg = KVA_TO_PA(allockernelpage());
        debugf("map halt %d, va:%p, pa:%p", i, va, newpg);
        kvmmmap(kpgtbl, va, newpg, PGSIZE, PTE_A | PTE_D | PTE_R | PTE_W);
    }
    c->sched_kstack_top = sched_stack + KERNEL_STACK_SIZE;
    // double the sched_stack to make a significant gap between different cpus.
    // if any kernel stack overflows, it will page fault.
}

```

```

    sched_stack += 2 * KERNEL_STACK_SIZE;
}

```

内核 scheduler 栈的结构如下所示，尽管所分配的页面在物理地址大概率是连续的，但是在虚拟地址上我们可以故意令它不连续。

```

[1ff], pte[0xffffffffc080400ff8]: 0xffffffffc0000000 -> 0x0000000080405000 -----V
[1f8], pte[0xffffffffc080405fc0]: 0xfffffffffff00000 -> 0x0000000080408000 -----V
[0], pte[0xffffffffc080408000]: 0xfffffffffff00000 -> 0x0000000080407000 DA--WRV
[1], pte[0xffffffffc080408008]: 0xfffffffffff001000 -> 0x0000000080409000 DA--WRV
[4], pte[0xffffffffc080408020]: 0xfffffffffff004000 -> 0x000000008040a000 DA--WRV
[5], pte[0xffffffffc080408028]: 0xfffffffffff005000 -> 0x000000008040b000 DA--WRV
[8], pte[0xffffffffc080408040]: 0xfffffffffff008000 -> 0x000000008040c000 DA--WRV
[9], pte[0xffffffffc080408048]: 0xfffffffffff009000 -> 0x000000008040d000 DA--WRV
[c], pte[0xffffffffc080408060]: 0xfffffffffff00c000 -> 0x000000008040e000 DA--WRV
[d], pte[0xffffffffc080408068]: 0xfffffffffff00d000 -> 0x000000008040f000 DA--WRV

```

- 最后，计算系统中所有应该被 Direct Mapping 管理的页面，即 `kernel_image_end_2M` 开始到物理内存空间的结束，从 `0xffffffffc0_80400000 -> 0x00000000_80400000` 处开始映射。注意到 `boot-stage page allocator` 所用的区域（`[0x8040_0000, 0x8060_0000)`）也会在这个映射范围之内。

之后，将剩下的空间交给 `kalloc.c` 中的 `kpgmgr` 管理，注意 `boot-stage page allocator` 的区域不在 `kpgmgr` 管理范围内：

```

// So page allocator should starts after these used pages.
kpage_allocator_base = init_page_allocator;
kpage_allocator_size = available_mems - (init_page_allocator - init_page_allocator_base);

```

kvmmap

`kvmmap` 负责按照 Sv39 的三级页表格式对 `vaddr` 逐级展开，分配相应的物理页作为页表，并填充对应的页表项。当遇到未被分配的次级页表时，从 `allockernelpage` 处分配一个，并设置上级到次级的 PTE (`RWX=000`) 即可。最后，在最后一级按照给定的 `perm` 设置 PTE：`pgtbl_level0[vpn0] = MAKE_PTE(paddr, perm)`。

当然，当我们映射一大片对齐到 2MiB 的内存区域时，我们可以使用一个 2MiB 的大页映射，而不是多个 4KiB 的映射。这需要满足大页映射的条件：`IS_ALIGNED(vaddr, PGSIZE_2M) && IS_ALIGNED(paddr, PGSIZE_2M) && sz >= PGSIZE_2M`。

```

void kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm) {
    assert(PGALIGNED(va));
    assert(PGALIGNED(pa));
    assert(PGALIGNED(sz));

    debugf("va:%p, pa:%p, sz:%x", va, pa, sz);

    pagetable_t __kva pgtbl_level1, pgtbl_level0;
    uint64 vpn2, vpn1, vpn0;

```

```

uint64 __kva vaddr      = va;
uint64 __kva paddr      = pa;
uint64 __kva vaddr_end = vaddr + sz;

while (vaddr < vaddr_end) {
    // try to add mapping: vaddr -> pa
    vpn2 = PX(2, vaddr);
    vpn1 = PX(1, vaddr);
    vpn0 = PX(0, vaddr);

    if (!(kpgtbl[vpn2] & PTE_V)) {
        // kpgtbl[vpn2] is not a valid PTE, allocate the level 1 pagetable.
        uint64 __kva newpg = allockernelpage();
        memset((void *)newpg, 0, PGSIZE);
        pgtbl_level1 = (pagetable_t)newpg;
        kpgtbl[vpn2] = MAKE_PTE(KVA_TO_PA(newpg), 0);
    } else {
        pte_t pte = kpgtbl[vpn2];
        // check validity: pte must points to next level page table.
        if ((pte & PTE_R) || (pte & PTE_W) || (pte & PTE_X))
            panic("kvmmmap: vaddr %p already mapped at level 2", vaddr);
        pgtbl_level1 = (pagetable_t)PA_TO_KVA(PTE2PA(kpgtbl[vpn2]));
    }
    if (!(pgtbl_level1[vpn1] & PTE_V)) {
        // pgtbl_level1[vpn1] is not a valid PTE.
        // try to allocate 2M page
        // , or allocate the level 1 pagetable.
        if (IS_ALIGNED(vaddr, PGSIZE_2M) && IS_ALIGNED(paddr, PGSIZE_2M) && sz >= PGSIZE_2M) {
            // it's ok for a huge page.
            pgtbl_level1[vpn1] = MAKE_PTE(paddr, perm);
            vaddr += PGSIZE_2M;
            paddr += PGSIZE_2M;
            sz -= PGSIZE_2M;
            continue;
        }
        uint64 __kva newpg = allockernelpage();
        memset((void *)newpg, 0, PGSIZE);
        pgtbl_level0 = (pagetable_t)newpg;
        pgtbl_level1[vpn1] = MAKE_PTE(KVA_TO_PA(newpg), 0);
    } else {
        pte_t pte = pgtbl_level1[vpn1];
        // check validity: pte must points to next level page table.
        if ((pte & PTE_R) || (pte & PTE_W) || (pte & PTE_X))
            panic("kvmmmap: vaddr %p already mapped at level 1", vaddr);
        pgtbl_level0 = (pagetable_t)PA_TO_KVA(PTE2PA(pgtbl_level1[vpn1]));
    }
    // check validity: pte must points to next level page table.
    if (pgtbl_level0[vpn0] & PTE_V)
        panic("kvmmmap: vaddr %p already mapped at level 0", vaddr);
    pgtbl_level0[vpn0] = MAKE_PTE(paddr, perm);
    vaddr += PGSIZE;
    paddr += PGSIZE;
    sz -= PGSIZE;
}
assert(vaddr == vaddr_end);
assert(sz == 0);
}

```

1.1.12 kalloc 模块

`kalloc.c` 会在启动后接管 Direct Mapping 区域的物理页面，其负责两个功能：

1. 分配物理页面：`void *__pa kallocpage()`, `void kfreepage(void *__pa pa)`
2. 对固定类型对象的动态分配和回收 (对象分配器管理): `allocator_init`, `kalloc`, `kfree`。

在 `kalloc` 接管内核启动后剩余的物理内存（即上图紫色部分）后，我们需要从它分配：

1. 每个 object allocator（对象分配器）所用的内存池
2. 每个 process 的 kernel stack
3. 每个 cpu scheduler 的 kernel stack

随后，用户空间所需要的页面，和配置页表所需要的页面均由 `kalloc` 模块管理。

1.1.13 实验部分思考题

注：这部分不要求在课堂实验报告上提交。

Question

我们在 `kernel.ld` 里面为 `_entry` 符号指定的虚拟地址是 `0xffff_ffff_8020_0000`，`entry.S` 中使用如下代码进行跳转

```

code:riscv:RISCV
•→ 0x80200000 <skernel+0000> auipc sp, 0xac
    0x80200004 <skernel+0004> mv     sp, sp
    0x80200008 <skernel+0008> auipc ra, 0x2
    0x8020000c <skernel+000c> jalr   488(ra)
source:os/entry.S+4
3 _entry:
→ 4     lla sp, boot_stack_top
  5     call bootcpu_entry

```

请思考：为什么在程序运行在 `0x0000_0000_8020_0000` 的地址上时，使用 `lla` 加载符号 `boot_stack_top` 和 `call bootcpu_entry` 跳转时 **能找到正确的物理地址 `0x8020_416c`**，而不是在此时为非法的虚拟地址 `0xffff_ffff_8020_416c`？

Hint: 我们是如何寻址的？回忆计算机组成原理课上学习的寻址模式。

页表与内存保护

在 `main.c` 的 `bootcpu_init` 函数中，`infof("start scheduler!");` 前插入以下代码：

```
static volatile uint64 number = 0xdeadbeefaabbccdd;
printf("addr of number: %p\n", &number);
printf("value of number: %p\n", number);
*(uint64*)&number = 0xdeadbeef00100073; // 00100073 is an ebreak instruction.
printf("changed value of number: %p\n", number);
((void(*)())&number)(); // execute a function, which address is &number
panic("qwq");
```

`make run` 运行内核，你应该会遇到 Kernel Panic, Code = 12, 查阅手册理解为什么会产生该异常。

回到上次lab的实验目录 `xv6lab3`，`git reset --hard da5eb84e`，然后做出一样的修改，`make clean && make run` 运行内核，你应该还会遇到 Kernel Panic，但是 Code = 3，这表示 `ebreak` 指令被执行了。

这即是在同一特权级下使用页表进行内存保护的意义：它可以阻止执行预期之外的行为。

如果将 `number` 的类型修改为 `const static volatile`，你应该会收获一个 Code = 15 的 Kernel Panic，这是因为 `const` 修饰符会使得它被分配在 `.rodata` 段，而这个段在页表映射中是 `R--` 的。

类似的，你也可以尝试修改 `scheduler` 函数的内容：

```
*(uint32*)&scheduler = 0x00100073; // 00100073: ebreak
infof("start scheduler!");
scheduler();
```

你可以对比在两次 lab 的代码仓库下，这样的行为会有怎样的差距。